

Pipelines entwickeln (Pipeline-Editor, Einstellungen)

Section 3 · Data Transformation and Modelling · Quelle: Kurs 3, Kapitel 4–5

Um eine Lakeflow Declarative Pipeline zu entwickeln, kommt der neue **Lakeflow Pipelines Editor** (Multi-File-Editor) zum Einsatz. Er bündelt Codebearbeitung, Pipeline-Einstellungen, Graph-Visualisierung und Ausführungsdetails in einer einzigen Oberfläche – statt, wie früher üblich, ein einzelnes großes Notebook zu pflegen, wird das Pipeline-Projekt als Satz einzelner Dateien organisiert.

1. Der Multi-File-Editor

Statt eines einzelnen monolithischen Notebooks besteht ein modernes Pipeline-Projekt aus mehreren `.sql` - oder `.py` -Dateien (Notebooks werden weiterhin unterstützt, sind aber nicht mehr die empfohlene Struktur). Diese Dateien sind im **Pipeline-Assets-Browser** sichtbar und lassen sich einzeln bearbeiten, während Code in einer Datei problemlos auf Tabellen oder Views verweisen kann, die in einer anderen Datei definiert wurden. Der Editor bietet außerdem:

- Eine interaktive **Pipeline-Graph**-Ansicht mit Datensatz-Abhängigkeiten und aktuellem Status.
- **Datenvorschauen** für Streaming Tables und Materialized Views direkt im Editor.
- **Ausführungseinblicke** (Execution Insights) und die Möglichkeit einer selektiven Ausführung einzelner Teile der Pipeline.
- Integrierte **Git**-Anbindung für Versionskontrolle.

Ein Pipeline-Projekt lässt sich sowohl über den Dateibrowser (**Create** → **ETL Pipeline**) als auch direkt über die zentrale **Jobs & Pipelines**-Ansicht anlegen.

Lakeflow Pipelines Editor  [Send feedback](#)

Purpose-built IDE for declarative data pipelines. Designed to support everything you need for building pipelines in one place: code-first authoring, folder-based organization, selective execution, data previews, and pipeline graphs. Integrated with the Databricks Platform, supporting version control, code reviews, and scheduling. Try it now. You can switch back anytime.

On 

Der Lakeflow Pipelines Editor vereint Code, Pipeline-Graph und Ausführungsdetails in einer Oberfläche.

2. Typische Ordnerstruktur eines Pipeline-Projekts

Ein Pipeline-Projekt organisiert seinen Code üblicherweise in mehreren Unterordnern, wobei genau festgelegt werden kann, welche davon tatsächlich zur Pipeline gehören:

Ordner	Zweck
exploration	Explorative Notebooks – typischerweise bewusst von der Pipeline ausgeschlossen
orders (Beispiel)	Enthält den eigentlichen Pipeline-Code, z. B. <code>orders_pipeline.sql</code>
python_excluded (Beispiel)	Alternative Implementierung, z. B. in Python statt SQL, ebenfalls ausgeschlossen

Über einen Rechtsklick auf einen Ordner lässt sich die Option **Include folder as pipeline source code** umschalten – so bestimmt man präzise, welche Dateien tatsächlich als Pipeline ausgeführt werden und welche (z. B. Explorationen) außen vor bleiben.

3. Wichtige Pipeline-Einstellungen

Über das Zahnrad-Symbol im Editor öffnet sich das Einstellungsfenster der Pipeline mit folgenden zentralen Bereichen:

- **Compute** – Serverless (empfohlen) oder klassische, fest dimensionierte Cluster.
- **Code Assets** – das Root-Verzeichnis der Pipeline („Pipeline Root Folder“, kann ein Git-Ordner sein) sowie die konkreten Quellcode-Pfade, die tatsächlich ausgeführt werden.
- **Default location for data assets** – Standard-Katalog und -Schema, in die neue Tabellen geschrieben werden (kann pro Datensatz überschrieben werden).
- **Konfigurationsparameter** – Schlüssel-Wert-Paare, die im Code referenziert werden können, um z. B. Pfade nicht hart zu codieren.
- **Erweiterte Einstellungen** – u. a. der *Channel* (Runtime-Version, meist „Current“) und optionale Event-Logs für Audit-Zwecke.

4. Eine einfache Pipeline aufbauen: Bronze, Silver, Gold

Das folgende Beispiel zeigt eine vollständige, dreistufige Pipeline im Medaillon-Muster: eine Streaming Table für die Rohdatenaufnahme, eine weitere Streaming Table für die Bereinigung und eine Materialized View für die Aggregation.

```

-- A. Bronze - Rohdatenaufnahme (Streaming Table)
CREATE OR REFRESH STREAMING TABLE orders_bronze
AS
SELECT *,
    current_timestamp() AS processing_time,
    _metadata.file_name AS source_file
FROM STREAM read_files(
    source_volume_path || '/orders',
    format => 'json'
);

-- B. Silver - Bereinigt & typisiert (Streaming Table)
CREATE OR REFRESH STREAMING TABLE orders_silver
AS
SELECT order_id,
    timestamp(order_timestamp) AS order_timestamp,
    customer_id,
    notifications
FROM STREAM orders_bronze;

-- C. Gold - Aggregation (Materialized View)
CREATE OR REFRESH MATERIALIZED VIEW gold_orders_by_date
AS
SELECT date(order_timestamp) AS order_date,
    count(*) AS total_daily_orders
FROM orders_silver
GROUP BY date(order_timestamp);

```

Bronze und Silver verwenden `STREAMING TABLE`, weil sie inkrementell verarbeiten sollen; Gold nutzt bewusst eine `MATERIALIZED VIEW`, da `GROUP BY`-Aggregationen nicht rein inkrementell berechnet werden können – Databricks optimiert die Neuberechnung dort, wo möglich, automatisch.

5. Dry Run, Ausführung und Historie

Vor der eigentlichen Ausführung lässt sich per **Dry Run** prüfen, ob der Pipeline-Code syntaktisch korrekt ist – ohne dass dabei Tabellen tatsächlich erstellt oder aktualisiert werden. Beim regulären Run bietet die Oberfläche zwei Modi:

Update-Typ	Materialized View	Streaming Table
Run pipeline (Refresh)	Aktualisiert Ergebnisse für die aktuelle Abfrage; inkrementell, sofern kosteneffizienter	Verarbeitet nur neue Datensätze über Flows

Run pipeline with full table refresh	Vollständige Neuberechnung	Leert Daten und Checkpoints, verarbeitet alle Quelldaten neu
---	----------------------------	--

Jeder inkrementelle Lauf lässt sich über `DESCRIBE HISTORY` nachvollziehen: In der Spalte `operation` erscheint bei Streaming Tables `STREAMING UPDATE` statt eines klassischen `WRITE` oder `MERGE`, und `operationMetrics` zeigt exakt, wie viele Zeilen im jeweiligen Lauf neu hinzugekommen sind – ein direkter Beleg dafür, dass wirklich nur die neuen Daten verarbeitet wurden.

Ergänzt aus der offiziellen Databricks-Dokumentation: Der Lakeflow Pipelines Editor wurde auf dem Data + AI Summit 2025 vorgestellt und ersetzt die frühere notebookzentrierte Entwicklungsweise. Laut aktueller Dokumentation liegen Quelldateien standardmäßig im Ordner `transformations` des Pipeline-Assets-Browsers und können Python- (`.py`) oder SQL-Dateien (`.sql`) sein – auch eine Mischung beider Sprachen innerhalb eines Projekts ist möglich, wobei Code in einer Datei problemlos auf Tabellen verweisen kann, die in einer anderen Datei definiert sind.
Quelle: [Develop and debug ETL pipelines with the Lakeflow Pipelines Editor – Databricks-Dokumentation](#)